TBZ · MODUL 321 · LB2 · EINZELABGABE TESTING

Testing Booking Service

Testdefinition · Testprotokoll · Automatisierte Tests

Modul	321 – Verteilte Systeme programmieren
Klasse	AP23a
Autor	Karol Krawiec
Komponente	Booking Service (Distributed Event Ticketing System)
Lehrperson	Stefan Kemper
Datum	02.11.2026
Code	github.com/AriesDevIo/Modul321AndrinKarolMarvin `booking-service/test/`, `scripts/ha-tests.sh`, Logs in `docs/test-logs/`

The document is written in English, like the project documentation.

Table of contents

1 **Scope and goals**

2 **Test environment**

2.1 Automated tests (component level)

2.2 System and HA tests

3 **Test strategy**

4 **Test cases**

4.1 Unit tests (test/unit.test.js)

4.2 Integration tests (test/bookings.test.js)

4.3 System and HA tests (LernMAAS VM)

5 **Test protocol**

5.1 Automated tests

5.2 System tests T1–T8

5.3 HA tests T9–T13

5.4 Findings

6 **Conclusion**

1 Scope and goals

This document covers the testing of my component, the **Booking Service**, and the system and high-availability tests that depend on it. It contains the three optional parts of the assignment:

Part	Section
Test definition (test cases and test environment)	2–4
Test protocol	5
Automated tests	3, 5.1 and `booking-service/test/`

What has to be shown:

1. The booking saga ends in a consistent state in **every** outcome: success, sold out, declined payment, and outages of the Event, Payment or Notification service or of the database.
2. **No overselling** under parallel load.
3. Only authorized users can book, read and cancel, and only their own bookings.
4. Invalid input is rejected before anything is stored.
5. The system as a whole keeps working when single instances fail (HA measures).

2 Test environment

2.1 Automated tests (component level)

Item	Value
Test runner	Node.js built-in test runner `node:test` (no extra dependencies), assertions with `node:assert/strict`
Coverage	Node.js built-in coverage (`--experimental-test-coverage`), limited to `src/**`
Runtime	Node.js 25.9 (macOS); the service itself runs on Node 20 in Docker, the tests use no newer APIs in the service code
Database	**Real PostgreSQL 16** in a throwaway Docker container on port 55433, database `bookings_test` (`test/test-db.sh`)
Other services	**Mock gateway** (`test/helpers.js`): an in-process HTTP server that implements `reserve` , `release` , `payments` and `notifications` with an in-memory inventory. Every endpoint can be switched to a failure mode: `error` (HTTP 500), `down` (connection dropped), `slow` (never answers, triggers the client timeout). All calls are recorded, so tests can check payloads and headers.
Isolation	Before every test the table is truncated and the mock gateway is reset. Tests run one file at a time (`--test-concurrency=1`) because they share the database.
Timeout	`UPSTREAM_TIMEOUT_MS=300` in tests, so timeout tests finish quickly

Why a real database and not a mock? The most important guarantees of the service (atomic status changes when cancelling, constraints, migration with an advisory lock, parallel requests) live in SQL. A mocked database would not test them.

How to run:

```
cd booking-service
npm install
./test/test-db.sh start      # PostgreSQL for the tests
npm test                     # all tests
npm run test:coverage        # tests + coverage report
./test/test-db.sh stop
```

2.2 System and HA tests

Item	Value
Environment	LernMAAS VM `m321-20-AP23a` (10.4.31.20), Ubuntu 24.04, Docker 29.1.3, Docker Compose 2.40.3
System	Full stack from `main` / `v1.0`: 4 services × 2 replicas, nginx gateway on port 8080, PostgreSQL 16
Client	macOS over the LernMAAS WireGuard VPN, `curl` + `jq`
Fault injection	Over SSH on the VM: `docker stop/start`, `kill -9` on the container process, renaming a table, `docker compose down/up`
Test data	Dedicated test users (`uiorg_*`, `uicust_*`) and events, deleted after the run

3 Test strategy

Level	What	Where	Automated
Unit	Configuration validation, JWT and role middleware	`test/unit.test.js`	yes
Integration (component)	All HTTP endpoints of the Booking Service against a real database and the mock gateway, including every failure path of the saga	`test/bookings.test.js`	yes
System (end to end)	Complete flows through the gateway with all real services (T1–T8)	LernMAAS VM	scripted (`curl`)
HA / fault injection	Replica failures, crashes, outages, restart (T9–T13)	LernMAAS VM	scripted (`ha-tests.sh`)

Test design techniques:

- **Equivalence classes and boundary values** for the input (quantity 0 / 1 / 10 / 11, non-integer, wrong types, empty token).
- **One test per saga outcome**, derived from the sequence diagram of the booking flow: every arrow that can fail has a test.
- **Fault injection** for every external dependency (Event, Payment, Notification service, database) in each failure mode (error, connection lost, timeout).
- **Concurrency tests** for the guarantees that only matter under parallel load (overselling, double cancel, parallel migration).

4 Test cases

4.1 Unit tests (`test/unit.test.js`)

ID	Test case	Expected result
BS-U1	Start without `JWT_SECRET`, `SERVICE_TOKEN` or `DB_PASSWORD` (3 cases)	Process exits with "Missing required environment variable …"
BS-U2	Start with only the required variables	Defaults: port 3000, gateway `http://gateway/api`, timeout 5000 ms, DB `bookings_db`
BS-U3	Valid JWT	`next()` is called, `req.user = {userId, username, role}` (no extra claims)
BS-U4	Missing header, wrong scheme, empty token, garbage token (4 cases)	`401`, `next()` not called
BS-U5	Expired JWT	`401`
BS-U6	`requireRole('customer')` with customer / organizer	passes / `403`

4.2 Integration tests (`test/bookings.test.js`)

Precondition for all: empty `bookings` table; mock event 1 with 10 tickets at CHF 49.90; customer *anna* (id 2), customer *ben* (id 3), an organizer (id 1).

ID	Test case	Expected result
Health and routing		
BS-01	`GET /health`	`200 {status: "ok"}`
BS-02	Unknown route	`404` JSON
Authentication and authorization		
BS-03	Book without token	`401`
BS-04	Token signed with another secret	`401`
BS-05	Organizer books	`403`, no other service is called
Input validation		

ID	Test case	Expected result
BS-06a–g	Missing body; quantity 0, 11, 1.5; eventId `"1"`, `-1`; empty paymentToken	`400`, nothing stored
BS-07	Malformed JSON	`400 "Invalid JSON body"`
BS-08	Quantity 1 and 10 (boundaries)	`201`
Booking saga		
BS-09	Book 2 tickets with `tok_ok`	`201 CONFIRMED`, `totalPrice` 99.80, stored as `CONFIRMED`, inventory 10 → 8
BS-10	Payloads of internal calls	reserve `{quantity: 2}`, payment `{bookingId, amount: 99.8, paymentToken}`, every call carries the service token
BS-11	Notification after success	one notification for user 2 that mentions the booking id
BS-12	Price 0.10 × 3	`totalPrice` 0.30 (no floating-point error)
BS-13	`tok_fail`	`402`, `FAILED`, one `release`, inventory back to 10
BS-14	Not enough tickets	`409`, `FAILED`, no payment, no release
BS-15	Unknown event	`404`, `FAILED`
BS-16	Event service unreachable	`503`, `FAILED`, no release
BS-17	Event service answers 500	`503`, `FAILED`
BS-18	Payment service answers 500	`503`, tickets released
BS-19	Payment service unreachable	`503`, tickets released
BS-20	Payment service hangs	`503` after the timeout (< 2 s), tickets released
BS-21	Notification service down	`201 CONFIRMED` (graceful degradation)
BS-22	Release fails after declined payment	still `402 FAILED`, error is logged
BS-23	**50 parallel single-ticket bookings, 10 tickets**	exactly 10 × `201`, 40 × `409`; DB: 10 `CONFIRMED`, 40 `FAILED`; inventory 0
Reading bookings		
BS-24	`GET /bookings`	only own bookings, newest first

ID	Test case	Expected result
BS-25	`GET /bookings/{id}` own booking	`200`
BS-26	Other user's booking, `abc`, unknown id	`404`
Cancelling		
BS-27	Cancel confirmed booking	`200 CANCELLED`, tickets released
BS-28	Cancel twice	second `409`, released only once
BS-29	3 parallel cancels of the same booking	exactly one `200`, released only once
BS-30	Cancel other user's / failed / unknown / invalid id	`404` / `409` / `404` / `404`
BS-31	Release fails during cancel	`503`, booking stays `CONFIRMED`
Database failures		
BS-34	Database down	`/health` → `503`
BS-35	Database fails after the reservation	`500`, tickets released
BS-36	Database down during read and cancel	`500` (no crash); service recovers afterwards
BS-37	Database not ready at startup	startup retries until reachable
BS-38	Database never reachable	startup gives up after the configured retries
Database		
BS-32	3 parallel migrations (replicas starting at once)	no error (advisory lock)
BS-33	Insert quantity 11 directly in SQL	rejected by the `CHECK` constraint

4.3 System and HA tests (LernMAAS VM)

ID	Test case	Expected result
T1	Register + log in	`201`, token
T2	Customer creates an event	`403`
T3	`reserve` without service token	`401`

ID	Test case	Expected result
T4	Book with `tok_ok`	`201 CONFIRMED`, `available` reduced, notification exists
T5	Book with `tok_fail`	`402`, `available` unchanged
T6	Book more than available	`409`
T7	Cancel	`CANCELLED`, tickets released
T8	Capacity 10, 50 parallel bookings	exactly 10 × `201`, `available = 0`
T9	Stop replica 1 of every service, then log in and book 10 times	all succeed
T10	A service process crashes	Docker restarts the container automatically, it becomes healthy again
T11	Both payment replicas down, then book	`503`, `available` unchanged
T12	Notifications can't be stored, then book	booking still `CONFIRMED`
T13	`docker compose down` + `up`	all data still there

5 Test protocol

5.1 Automated tests

Date	02.11.2026
Environment	as in 2.1 (Node.js 25.9, PostgreSQL 16 in Docker)
Command	`npm run test:coverage`
Result	**55 of 55 passed**, 0 failed, duration ≈ 2 s

All test cases from 4.1 and 4.2 passed. Full output: `docs/test-logs/booking-service-tests.txt`.

Coverage (`src/**`):

File	Lines	Branches	Functions
`app.js`	100 %	100 %	100 %
`auth.js`	100 %	100 %	100 %
`bookings.js`	100 %	97.1 %	92.3 %
`clients.js`	100 %	100 %	83.3 %
`config.js`	100 %	100 %	100 %
`db.js`	100 %	100 %	75 %
Total	**100 %**	**98.2 %**	**90 %**

The missing function coverage comes from small inline fallback callbacks, for example `.catch(() => ({}))` for a response that is not JSON (`clients.js`) and `.catch(() => {})` around the advisory unlock (`db.js`). They only run in rare situations that would need extra fault injection and contain no logic. `src/index.js` (process start and shutdown) is not part of the automated tests; it is covered by the system tests, where the service starts in Docker.

The first version of the suite (50 tests) reached 94 % line coverage. The uncovered lines were all database error paths; tests BS-34 to BS-38 were added for them.

5.2 System tests T1–T8

Date	26.10.2026
Environment	LernMAAS VM, full stack (2.2)

ID	Result	Observed
T1	✅	`201` / `201`, token received
T2	✅	`403`
T3	✅	`401`
T4	✅	`CONFIRMED`, `available` 10 → 8, 1 notification
T5	✅	`402`, `available` 8
T6	✅	`409`
T7	✅	`CANCELLED`, `available` 10
T8	✅	10 × `201`, 40 × `409`, `available = 0`

5.3 HA tests T9–T13

Date	02.11.2026, 08:28–08:31
Environment	LernMAAS VM, full stack (2.2)
Evidence	`docs/test-logs/ha-tests-vm.txt`

ID	Result	Observed
T9	✅	`user-`, `event-`, `booking-`, `payment-service-1` stopped. Login `200`, 10 bookings → 10 × `201`, `available` 100 → 90.
T10 (1st attempt, `docker kill`)	❌	Container stayed `exited`, `RestartCount 0`.
T10 (retest, `kill -9` on the process)	✅	`RestartCount` 0 → 1 after 2 s (new PID), `healthy` again after ≈ 6 s.
T11	✅	Both payment replicas stopped: booking `503`, `available` 89 → 89. After restart: booking `201`.
T12	✅	Table `notifications` renamed: booking `CONFIRMED`, no new notification, Booking Service logs `notification returned 500`. Table restored afterwards.
T13	✅	`docker compose down` + `up`: 10/10 containers healthy after ≈ 15 s, customer's bookings 14 → 14, event inventory unchanged.

5.4 Findings

#	Finding	Assessment / action
1	**`docker kill` does not trigger `restart: always`.** Docker treats `docker kill` (like `docker stop`) as a manual stop and deliberately does not restart. A real crash (process killed with `kill -9`, as in the retest) is restarted within 2 s.	Not a defect of the system, but of the original test procedure. The README's live demo used `docker kill`; for the demo the crash must be simulated with `sudo kill -9 $(docker inspect -f '{{.State.Pid}}' <container>)`. Test T10 updated accordingly.
2	**First request after a replica fails takes ≈ 3 s.** nginx first tries the stopped replica and waits for `proxy_connect_timeout` (3 s) before failing over; the following requests are fast (≈ 20 ms) because the replica is marked as failed for 10 s.	Acceptable (no request fails). Could be improved with a shorter connect timeout (e.g. 1 s) or active health checks.
3	During T13, one inventory read issued right before `docker compose down` returned no JSON. It could not be reproduced in 6 further reads with a stopped replica (all `200`).	Most likely the request hit the moment `event-service-2` was being torn down after finding 1. Not reproducible; reads during replica failures are covered by finding 2 and T9.
4	**Known limitation, not covered by tests:** if `reserve` times out *after* the Event Service has already reserved the tickets, the Booking Service marks the booking `FAILED` without releasing, so those tickets stay blocked.	Documented in the conclusion of the project documentation. Fix: cleanup job (F10) or idempotent reservations per booking id.

6 Conclusion

All 55 automated tests and all 13 system and HA tests passed (T10 after correcting the test procedure). The automated tests cover 100 % of the lines and 98 % of the branches of the Booking Service and run in about two seconds, so they can be run before every commit. Writing the failure tests found no bug in the service, but the HA tests found a mistake in our planned demo (finding 1), which would otherwise have failed during the presentation.

Next steps would be to run the tests automatically on every push (e.g. GitHub Actions with a PostgreSQL service container) and to add a test for finding 4 once the cleanup job exists.